



Name: Wakishta W Dissanayake

Student Reference Number: 10953506

Module Code: PUSL3190	Module Name: Computing Project
Coursework Title: Project Initiation Document (PID)	
Deadline Date:16/01/2026	Member of staff responsible for coursework: Mr. Diluka Wijesinghe
Programme: BSc.(Hons) Computer Science	
Please note that University Academic Regulations are available under Rules and Regulations on the University website www.plymouth.ac.uk/studenthandbook .	
Group work: please list all names of all participants formally associated with this work and state whether the work was undertaken alone or as part of a team. Please note you may be required to identify individual responsibility for component parts. <i>We confirm that we have read and understood the Plymouth University regulations relating to Assessment Offences and that we are aware of the possible penalties for any breach of these regulations. We confirm that this is the independent work of the group.</i> Signed on behalf of the group:	
Individual assignment: <i>I confirm that I have read and understood the Plymouth University regulations relating to Assessment Offences and that I am aware of the possible penalties for any breach of these regulations. I confirm that this is my own independent work.</i> Signed :	
Use of translation software: failure to declare that translation software or a similar writing aid has been used will be treated as an assessment offence. I *have used/not used translation software.	

If used, please state name of software.....

Overall mark _____ **%** **Assessors Initials** _____ **Date** _____



PUSL3190 Computing Project

Project Initiation Document (PID)

Stock Price Indicator And Prediction Project

Supervisor: Diluka Wijesinghe

Name: Wakishta W Dissanayake

Plymouth Index Number: 10953506

Degree Program: BSc.(Hons) Computer Science

Table of Contents

CHAPTER 01: INTRODUCTION.....	6
1.1 BACKGROUND AND CONTEXT	6
1.2 PROBLEM STATEMENT.....	6
1.3 PURPOSE AND SCOPE	7
1.4 EXPECTED IMPACT AND STAKEHOLDERS.....	7
CHAPTER 02: BUSINESS CASE	9
2.1 BUSINESS NEED.....	9
2.2 BUSINESS OBJECTIVES	9
CHAPTER 03: PROJECT OBJECTIVES	10
3.1 FUNCTIONAL DELIVERABLES	10
3.2 QUALITY AND SUCCESS METRICS.....	10
CHAPTER 04: LITERATURE REVIEW	11
4.1 INTRODUCTION AND SEARCH STRATEGY	11
4.2 THEMATIC ORGANIZATION.....	11
4.3 CRITICAL ANALYSIS AND IDENTIFIED GAPS.....	11
4.4 IMPLICATIONS FOR DESIGN CHOICES	12
CHAPTER 05: METHOD OF APPROACH	13
5.1 RESEARCH & DEVELOPMENT DESIGN	13
5.2 DATA STRATEGY	14
5.3 IMPLEMENTATION STACK.....	14
5.4 EVALUATION STRATEGY	14
5.4 PROJECT MANAGEMENT METHODOLOGY.....	15
CHAPTER 06: CONCEPTUAL DIAGRAM	16
CHAPTER 07: INITIAL PROJECT PLAN	17
CHAPTER 08: RISK ANALYSIS	18
CHAPTER 09: REFERENCING	19

List of Tables and Figures

Figure 1- flow chart.....	13
Figure 2- conceptual diagram	16
Figure 3- Gantt chart.....	17
Table 1- sprint table	15
Table 2- Risk analysis.....	18

Chapter 01: Introduction

1.1 Background and Context

The modern financial landscape is characterized by extreme volatility and an overwhelming influx of data, making traditional market analysis increasingly difficult for the average participant. While institutional investors utilize high-frequency data and enterprise-grade AI, individual investors often rely on lagging indicators or manual observations. This project sits at the intersection of Financial Technology (FinTech) and Deep Learning, specifically focusing on the application of Long Short-Term Memory (LSTM) networks.

Stock prices are inherently time-series data, meaning their future values are influenced by historical sequences. Traditional models often fail because they treat these relationships as linear; however, deep learning models like LSTMs are designed to capture "long-term dependencies" the ability to remember and relate events from far in the past to the present moment. By transitioning these models from isolated research environments into a production-ready Next.js web application, this project seeks to democratize access to advanced predictive insights.

1.2 Problem Statement

The stock market is a highly dynamic and unpredictable environment where behavior is influenced by numerous factors that interact in complex, non-linear ways.

The Technical Gap: Traditional forecasting methods, such as ARIMA or simple Linear Regression, often struggle to capture these intricate patterns, leading to unreliable predictions.

The Accessibility Gap: In recent years, Deep Learning models have shown success, but they are typically limited to specialized Python environments like Jupyter notebooks, which are not accessible to everyday users.

The Cost Barrier: Individual investors and decision-makers face high-cost enterprise tools that remain out of reach for small investors or independent researchers.

The User Experience Gap: Many existing forecasting systems rely on outdated models and offer poor user experiences, lacking the interactivity required for modern financial analysis.

This project aims to bridge these gaps by creating a modern, user-friendly web platform that combines Python-based LSTM forecasting with a scalable and responsive Next.js frontend.

1.3 Purpose and Scope

The primary purpose of this project is to develop a robust, end-to-end stock price forecasting application delivered through a high-performance web interface. The project encompasses the entire lifecycle from data acquisition to cloud deployment.

Detailed Scope:

- **Data Acquisition and Preprocessing:** Collecting historical stock data from reliable APIs like Yahoo Finance. This involves cleaning, normalizing, and converting data into time-series sequences.
- **Feature Engineering:** Computing key technical indicators—including the Moving Average (MA), Relative Strength Index (RSI), and Moving Average Convergence Divergence (MACD)—to enhance the model's predictive capability.
- **Model Development:** Developing, training, and validating an LSTM model using Python and TensorFlow. The model will be fine-tuned through experimentation with hyperparameters to balance accuracy and efficiency.
- **System Architecture:** Implementing a decoupled microservice architecture where the Python ML Model is an independent RESTful service (using FastAPI) callable by the Next.js frontend.
- **Frontend Design:** Creating a responsive interface in Next.js that provides user authentication, ticker search, and interactive charting of predicted prices.
- **Deployment and MLOps:** Containerizing services using Docker for seamless deployment to cloud platforms like Vercel and AWS.
- **Limitations:** To maintain a realistic development plan, the project excludes real-time trading execution, integration with brokerage APIs, portfolio management, and macroeconomic factor analysis such as sentiment or news-based forecasting.

1.4 Expected Impact and Stakeholders

This project creates value by integrating advanced data science into an accessible web environment, empowering users to make more informed decisions.

Primary Stakeholder: The student developer, Wakishta W Dissanayake, whose goal is to fulfill the academic requirements of the PUSL3190 Computing Project.

Secondary Stakeholders:

- Academic Supervisors: To assess the technical and academic standard of the implementation.
- Individual Investors: Small-scale users who require affordable, cutting-edge predictive insights.
- Financial Researchers: Those interested in the practical application of MLOps and microservice architectures in finance.

Chapter 02: Business Case

2.1 Business Need

The project addresses a critical lack of accessibility in the FinTech space. While deep learning models like LSTMs have shown high success in time-series prediction, they remain locked within specialized Python environments like Jupyter Notebooks, which are inaccessible to the average user. Furthermore, existing individual-market tools often rely on outdated ARIMA models that struggle with market volatility. This project solves these issues by providing a scalable, cloud-ready, and user-friendly interface for advanced AI models.

2.2 Business Objectives

The business-level goals focus on the value provided to the end-user:

Democratize AI Insights: Provide a professional-grade forecasting tool to independent researchers at no cost.

Enhance Decision Speed: Deliver prediction results and visual charts in under 8 seconds to allow for timely analysis.

Improve Accuracy: Achieve a 95% or higher confidence in capturing market trends compared to baseline linear models.

Accessibility: Ensure 100% responsiveness across mobile and desktop devices to support on-the-go financial monitoring.

Chapter 03: Project Objectives

These objectives define the specific technical deliverables and success criteria:

3.1 Functional Deliverables

Trained LSTM Model: A validated deep learning model capable of processing multivariate time-series data.

ML Microservice: A standalone API built with FastAPI that performs inference and returns JSON-formatted predictions.

Web Dashboard: A Next.js application featuring secure JWT-based authentication and a ticker search system.

Interactive Visualization: Dynamic charts rendering both historical data and future "horizon" forecasts (e.g., 7-day predictions).

3.2 Quality and Success Metrics

Predictive Accuracy: The LSTM model must outperform an ARIMA baseline in terms of Root Mean Square Error (RMSE) and Mean Absolute Error (MAE).

Performance: Frontend-to-backend communication must be optimized to ensure low latency.

Scalability: The system must be fully containerized using Docker to allow for independent scaling of the frontend and the ML service.

Chapter 04: Literature Review

4.1 Introduction and Search Strategy

The purpose of this project is to develop an accessible, high-performance stock forecasting platform. The search strategy focused on identifying studies comparing statistical and deep learning models for time-series forecasting. Keywords used include "LSTM," "ARIMA," "Stock Price Prediction," and "Microservice Architecture". Databases such as IEEE Xplore, ResearchGate, and SpringerLink were utilized to find recent scholarly articles (2014–2025).

4.2 Thematic Organization

A. Traditional Statistical Models (ARIMA) Early research in stock forecasting heavily utilized the Autoregressive Integrated Moving Average (ARIMA) model. Studies like Ariyo et al. (2014) demonstrate that while ARIMA is effective for linear and stationary datasets, it struggles with the high volatility and non-linear nature of real-world financial markets.

B. Deep Learning and LSTM Networks Recent literature shows a shift toward Long Short-Term Memory (LSTM) networks to capture long-term dependencies in time-series data. Wu, Chen, and Ding (2023) and Yu (2025) highlight that LSTMs outperform traditional models by handling temporal relationships more effectively. Furthermore, integrating technical indicators like RSI and MACD into these models further enhances predictive accuracy.

C. System Deployment and Accessibility Most existing studies focus on the algorithmic performance within isolated Python environments (e.g., Jupyter notebooks). There is limited research on the practical deployment of these models as scalable web services.

4.3 Critical Analysis and Identified Gaps

Methodological Gap: While LSTMs are powerful, many implementations lack a direct comparison with conventional models like ARIMA within a single production-ready interface.

Technological Gap: Current solutions are often monolithic and not designed for web-based accessibility, making them difficult for everyday investors to use.

Architectural Gap: There is a clear absence of microservice-based solutions that decouple heavy machine learning computation (Python) from responsive user interfaces (Next.js).

4.4 Implications for Design Choices

This project addresses these gaps by implementing a decoupled microservice architecture. By deploying the LSTM model via a FastAPI backend and a Next.js frontend, the system will provide the high-performance forecasting identified in literature within a scalable, user-friendly platform.

Chapter 05: Method of Approach

5.1 Research & Development Design

The development process follows an iterative lifecycle designed to seamlessly integrate advanced machine learning with a responsive web interface, as detailed in the accompanying data processing flowchart. The journey begins with user interaction, where the system checks for active sessions or initiates FR1: User Authentication & JWT Generation to ensure secure access. Once authenticated, the user provides a stock ticker and forecast horizon (FR2), triggering a request through Next.js API Routes to the FastAPI ML Microservice. The backend machine learning pipeline then executes a sequence of critical operations: fetching historical data via the Yahoo Finance API, performing rigorous data cleaning and normalization, and calculating technical indicators such as RSI, MACD, and Moving Averages. These multivariate sequences are fed into the trained LSTM model to generate price predictions, which are then formatted as JSON and sent back to the frontend. Finally, the process concludes with FR3: Rendering Interactive Chart.js Visualizations, allowing the user to intuitively view and analyze the predicted financial trends.

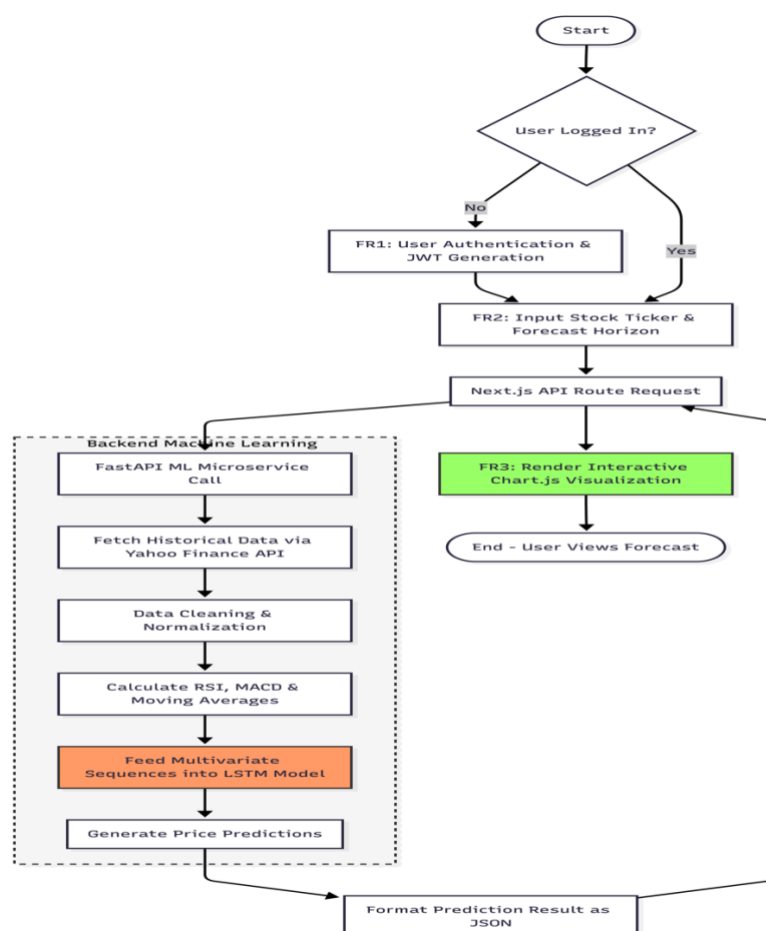


Figure 1- flow chart

5.2 Data Strategy

- Collection: 5-10 years of historical daily data will be pulled from Yahoo Finance for selected tickers.
- Preprocessing: Data will undergo Min-Max scaling, handling of missing values, and technical indicator calculation (MA, RSI, MACD).
- Splitting: Data will be divided into training (80%), validation (10%), and testing (10%) sets to ensure the model generalizes well to unseen data.

5.3 Implementation Stack

- Backend/AI: Python 3.x, TensorFlow/Keras (for LSTM), FastAPI (for the microservice).
- Frontend: Next.js (React framework), Tailwind CSS (styling), Chart.js (visualization).
- DevOps: Docker for containerization and GitHub Actions for version control.

5.4 Evaluation Strategy

The model will be evaluated using:

- RMSE (Root Mean Square Error): To penalize larger prediction errors.
- MAE (Mean Absolute Error): To understand the average magnitude of errors.
- Load Testing: Measuring system responsiveness under concurrent user requests.

5.4 Project Management Methodology

The project will follow an Agile methodology, specifically utilizing Sprints to ensure iterative development, flexibility, and continuous integration. This approach is ideal for managing the complexities of decoupling a machine learning (ML) microservice from a web frontend.

Sprint	Focus Area	Key Tasks & Deliverables
Sprint 1	Research & Data	Gathering requirements, collecting historical data via Yahoo Finance API, and performing data preprocessing/normalization
Sprint 2	ML Model Development	Designing the LSTM architecture, training with technical indicators (RSI, MACD), and validating against ARIMA
Sprint 3	Backend & API	Developing the FastAPI/Flask microservice, containerizing the model with Docker, and setting up the MongoDB schema
Sprint 4	Frontend & Integration	Building the Next.js interface, implementing user authentication, and establishing the secure API bridge to the ML service
Sprint 5	Testing & Deployment	Performing unit, integration, and performance tests, followed by cloud deployment to Vercel and AWS

Table 1- sprint table

Chapter 06: Conceptual Diagram

The conceptual architecture of the system is designed around a decoupled microservice strategy to ensure high performance and scalability. As illustrated in the high-level diagram, the Client Layer facilitates user interaction through a responsive Next.js Frontend. This communicates with the Web Application Layer, where Next.js API Routes manage secure authentication, ticker requests, and data persistence using MongoDB. To handle intensive computational tasks without impacting user experience, the ML Inference Layer utilizes a FastAPI/Flask Microservice that interacts with the Trained LSTM Model to process raw data and generate predictions. Finally, the External Data Layer ensures the system remains current by fetching real-time financial data from the Yahoo Finance API.

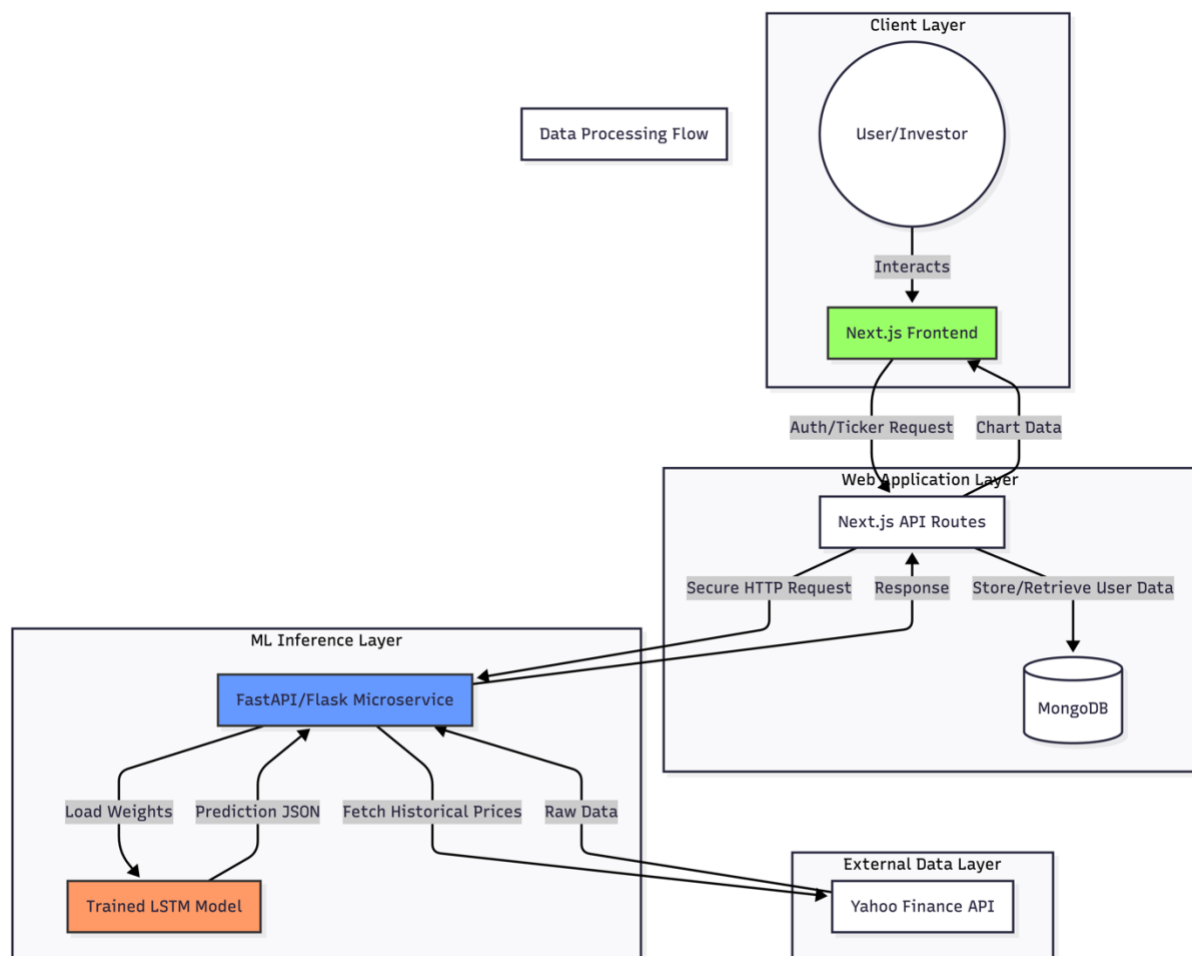


Figure 2- conceptual diagram

Chapter 07: Initial Project Plan

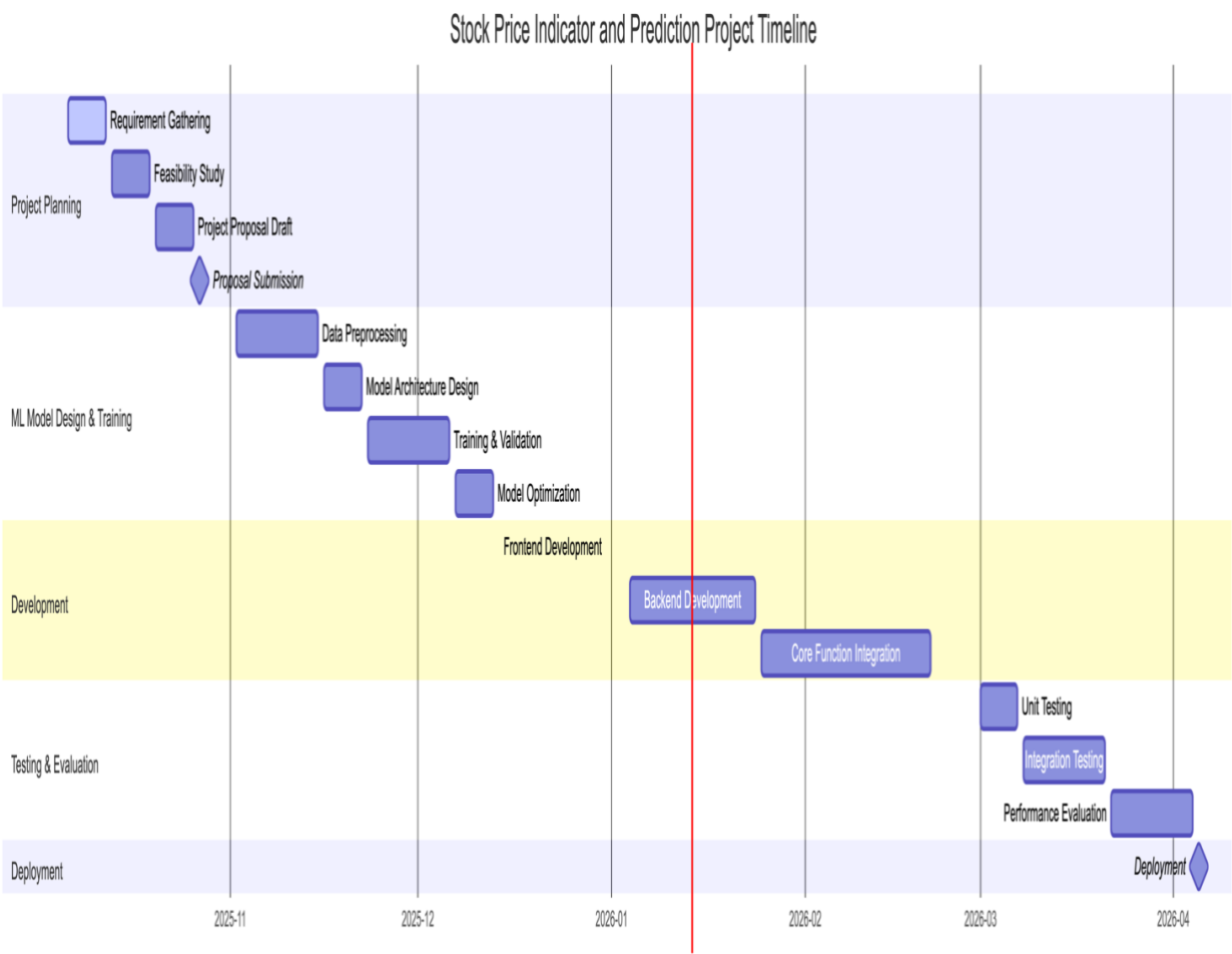


Figure 3- Gantt chart

Chapter 08: Risk Analysis

Based on the "Stock Price Indicator and Prediction" project and the PID template, the following Risk Analysis identifies critical technical, data-related, and operational risks. Each risk is assessed by its Likelihood (L) and Impact (I) on a scale of 1–5 (with 5 being the highest) and provides a proactive mitigation strategy.

Risk ID	Risk Description	L	I	Mitigation Strategy
R1	Model Overfitting: The LSTM model may "memorize" historical noise instead of general patterns, leading to poor live performance.	4	5	Implement Early Stopping , Dropout layers , and L2 Regularization during training. Validate using K-fold cross-validation on different time periods.
R2	API Latency: High response times from the Python ML microservice could delay chart rendering beyond the 8-second NFR.	3	4	Use FastAPI for its asynchronous capabilities. Implement caching for frequently requested tickers to reduce redundant computation.
R3	Data Source Volatility: Changes or outages in the Yahoo Finance API (External Data Layer) could break the data pipeline.	2	5	Implement robust exception handling in the data fetching script. Design a fallback mechanism to use local cached data if the API is unreachable.
R4	Deployment Incompatibility: Differences between local development and cloud environments (Vercel/AWS) may cause service failures.	3	4	Use Docker containerization for both the Next.js frontend and Python backend to ensure environment parity across all stages.
R5	Security Vulnerabilities: Unauthorized access to user accounts or data during transmission between microservices.	2	5	Enforce HTTPS for all API communication and implement JWT (JSON Web Tokens) for secure authentication between layers.

Table 2- Risk analysis

Chapter 09: Referencing

Indian stock market prediction using artificial neural networks on tick data Published: 21 March 2019. Available at <https://jfin-swufe.springeropen.com/articles/10.1186/s40854-019-0131-7>

AI Stock Market Prediction with Python and LSTM." (2025). Artificial Intelligence in Plain English. Available at <https://ai.plainenglish.io/ai-powered-stock-market-prediction-with-python-and-lstm-dd7c82024bdf>

Ariyo, A. A., Adewumi, A. O., & Ayo, C. K. (2014). "Stock Price Prediction Using the ARIMA Model." In Proceedings of the 2014 UKSim-AMSS 16th International Conference on Computer Modelling and Simulation. Available at <https://ieeexplore.ieee.org/document/7046047>

Sunki, A., et al. (2024). "Time series forecasting of stock market using ARIMA, LSTM and FB prophet." MATEC Web of Conferences. Available at https://www.researchgate.net/publication/379050041_Time_series_forecasting_of_stock_market_using_ARIMA_LSTM_and_FB_prophet

Wu, H., Chen, S., & Ding, Y. (2023). "Comparison of ARIMA and LSTM for Stock Price Prediction." Financial Engineering and Risk Management. Available at <https://www.scirp.org/reference/referencespapers?referenceid=3830243>

Yu, Y. (2025). "LSTM-Based Time Series Prediction Model: A Case Study with YFinance Stock Data." ITM Web of Conferences. Available at https://www.researchgate.net/publication/388315149_LSTM-Based_Time_Series_Prediction_Model_A_Case_Study_with_YFinance_Stock_Data

Li, Z., et al. (2023). "Stock Market Analysis and Prediction Using LSTM: A Case Study on Technology Stocks." Innovations in Applied Engineering and Technology. Available at https://www.researchgate.net/publication/379811995_Stock_Market_Analysis_and_Prediction_Using_LSTM_A_Case_Study_on_Technology_Stocks

<https://ai.plainenglish.io/ai-powered-stock-market-prediction-with-python-and-lstm-dd7c82024bdf>